TASK-9: Develop a basic Spring MVC application.

AIM:

To develop a basic MVC-based web application to display employee details using Node.js and MySQL. To handle user requests and dynamically show employee data on a web page.

ALGORITHM:

1. Create a MySQL database and employee table
2. Insert sample employee records into the table
3. Initialize a Node.js application using Express
4. Establish connection between Node.js and MySQL database
5. Create a route /employees to fetch employee data
6. Start the server on a specific port (e.g., 3000)
7. Design an HTML page with a button and table
8. When user clicks button, send request using JavaScript (fetch)
9. Receive employee data from backend in JSON format
10. Display the data dynamically inside the HTML table

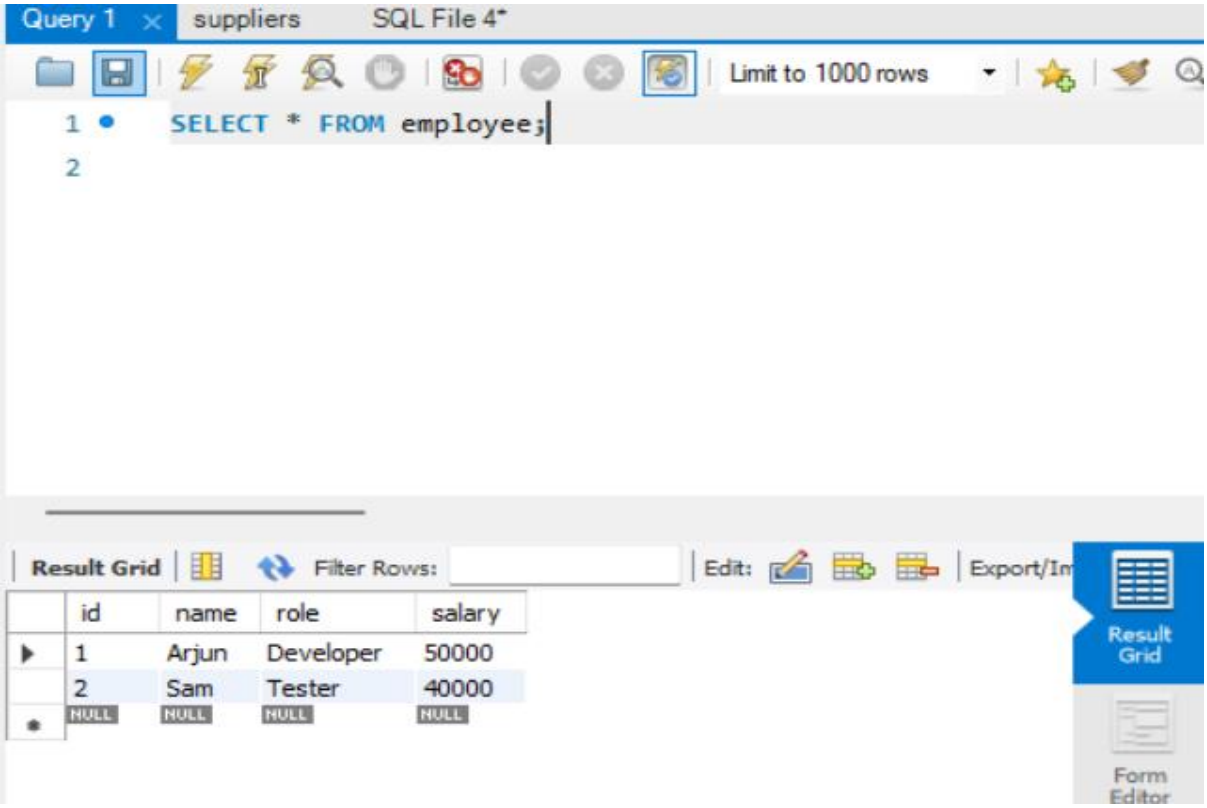
PROGRAM:

```
<!DOCTYPE html>
<html>
<head>
  <title>Employee List</title>
  <style>
    body {
      font-family: Arial;
      text-align: center;
    }
    table {
      margin: auto;
      border-collapse: collapse;
      width: 60%;
    }
    th, td {
```

```
            border: 1px solid black;
            padding: 10px;
        }
        th {
            background-color: lightblue;
        }
    </style>
</head>
<body>
<h2>Employee Details</h2>
<button onclick="loadEmployees()">Load Employees</button>
<table id="empTable">
    <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Role</th>
        <th>Salary</th>
    </tr>
</table>
<script>
function loadEmployees() {
    fetch("http://localhost:3000/employees")
    .then(res => res.json())
    .then(data => {
        let table = document.getElementById("empTable");
        // Clear old data
        table.innerHTML = `
            <tr>
                <th>ID</th>
                <th>Name</th>
                <th>Role</th>
```

```
            <th>Salary</th>
          </tr>
        `;
        if (data.length === 0) {
          alert("No employees found!");
          return;
        }
        data.forEach(emp => {
          let row = table.insertRow();
          row.insertCell(0).innerText = emp.id;
          row.insertCell(1).innerText = emp.name;
          row.insertCell(2).innerText = emp.role;
          row.insertCell(3).innerText = emp.salary;
        });
      })
      .catch(err => {
        console.error(err);
        alert("Error fetching data");
      });
    }
    </script>
    </body>
    </html>
```

Output:



Result:

Employee details are stored in the MySQL database and retrieved successfully using Node.js.
The data is displayed dynamically in the web page when the user clicks the button.The application works correctly following the MVC flow (Model–View–Controller).